CIFAR-10 Classification with SimpleCNN, SimpleViT and a CNN–ViT Composition

雷源泽　2510124204
Peking University

Abstract

We implement a *SimpleCNN* (85,642 parameters) and a from-scratch *SimpleViT* in PyTorch and train both on CIFAR-10 for 75 epochs with W&B tracking. The CNN reaches **79.48%** test accuracy and the ViT **65.01%**. The optional *SimpleComposition* (81,194 < 120K parameters) reaches **69.98%**.

1 Method

1.1 Data preparation

`prepare_data.py` loads the 50,000-image CIFAR-10 training set and splits the official 10,000-image test set into validation/test with an 8:2 ratio using `random_split` and a `torch.Generator` seeded with 0 (reproducible). All images use `ToTensor()` and normalisation with mean/std 0.5; training additionally uses `RandomCrop(32, padding=4)` and `RandomHorizontalFlip()`.

1.2 SimpleCNN

Two 3×3 convolutions (32/64 channels) with ReLU and 2×2 max-pooling (three pools in total: $32\to16\to8\to4$), flatten ($64\times4\times4=1024$), dropout, and two fully connected layers ($1024\to64\to10$). Parameter count: $896+18{,}496+65{,}600+650=85{,}642<100$K. Dropout is active in training and disabled by `model.eval()`. `forward_map()` exposes the 4×4 feature map for the bonus model.

1.3 SimpleViT

All blocks are implemented from scratch (no `nn.Transformer*`): a `FeedForward` block (LayerNorm–Linear–GELU–Dropout–Linear–Dropout); multi-head `Attention` with a fused QKV projection, scaled dot-product attention ($1/\sqrt{d_{head}}$), softmax, attention dropout and an output projection; a `TransformerEncoder` of pre-norm residual blocks

with a final LayerNorm; and `SimpleViT`, which patchifies the image with a strided convolution, adds a learnable class token and positional embeddings, applies embedding dropout, runs the encoder and classifies from the class token. Configuration: $dim = 64$, $depth = 4$, $heads = 4$, $dim_head = 16$, $mlp_dim = 128$, 4×4 patches.

1.4 Training setup

`train.py` fixes all seeds to 0, trains for exactly 75 epochs and saves the best-validation checkpoint (`state_dict`) plus the constructor hyper-parameters (`pickle`). CNN: SGD (lr 0.2, momentum 0.9, weight decay 5×10^{-4}); ViT/Composition: AdamW (lr 10^{-3}); all with `CosineAnnealingLR` and batch size 512. W&B logs the step-level training loss, the epoch-level training loss, validation loss and validation accuracy, and `test.py` reloads both checkpoint files and reports the test accuracy.

2 Results

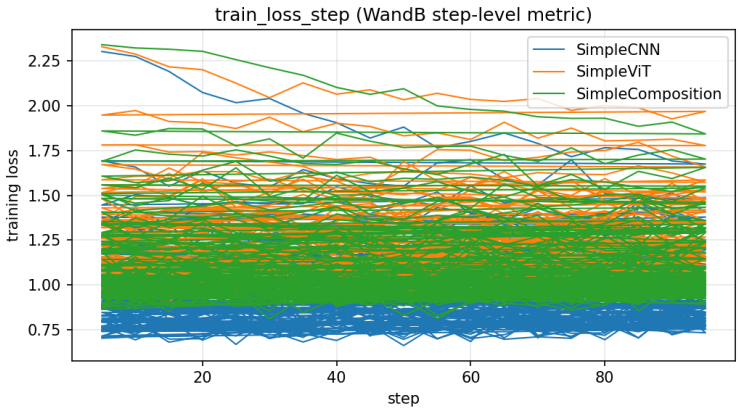


图 1: W&B `train_loss_step`: the CNN descends fastest to the lowest loss; the ViT plateaus higher; the Composition lies between.

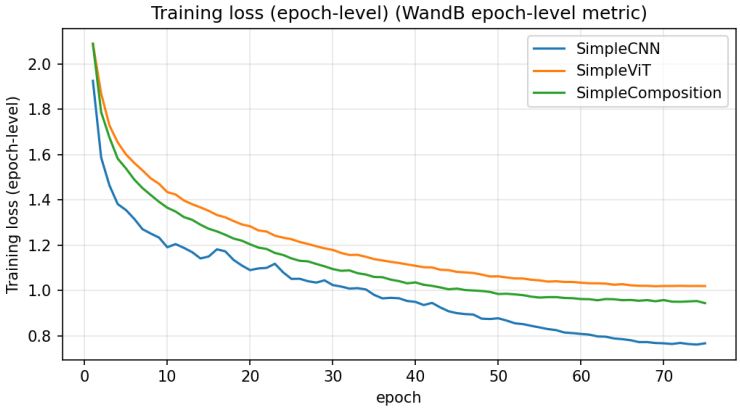


图 2: W&B `train_loss_epoch`: the ViT leaves more training loss, i.e. it underfits this small-data task without convolutional inductive bias.



图 3: W&B `val_loss`: the CNN generalises best; the ViT's higher loss is consistent with the expected weakness of Transformers on small datasets.

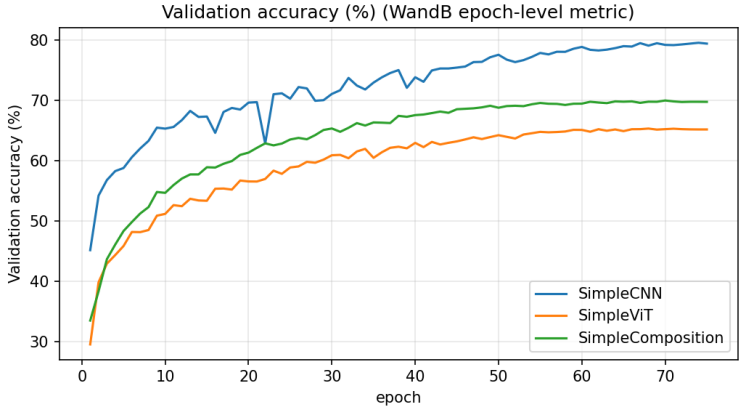


图 4: W&B `val_acc`: CNN vs. ViT vs. Composition on one axis. The CNN reaches 79.54% and is still improving when the cosine schedule ends, whereas the ViT saturates near 65.2%.

The four W&B metrics above (step loss, epoch loss, validation loss, validation accuracy) show a consistent picture: the CNN converges fastest and best, the plain ViT saturates early, and the Composition recovers part of the gap. This matches the handout's remark that removing the convolutional inductive bias hurts in the small-data/small-model regime. Final test accuracies: **CNN 79.48%**, **ViT 65.01%**, **Composition 69.98%**.

3 Bonus: SimpleComposition

`SimpleComposition` feeds the CNN feature maps (`forward_map`, $32\times4\times4$) into a small ViT ($patch = 2 \times 2$, $dim = 64$, $depth = 2$) and trains end-to-end. Under the course counting rule (CNN non-linear part + full ViT) it uses $5{,}088 + 76{,}106 = 81{,}194 < 120\text{K}$ parameters and reaches **69.98%** test accuracy (curve in Fig. 4), clearly above the plain ViT.

4 Conclusion

The compact CNN gives the best accuracy/size trade-off, the from-scratch ViT is limited by the small-data regime, and a CNN–ViT composition partially closes the gap. All runs are reproducible (fixed seeds, saved hyper-parameters, W&B logging).

References